

Trainer Guide: Java Collections, Class Relationships & File I/O

A slide-by-slide read-aloud script for the java-collections-theory.html deck. Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: java-collections-theory.html (24 slides)

How to use this guide: each slide below has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Slide 1 - Title: Collections, Class Relationships & File I/O

ON SCREEN

The blue title slide: the session name, and a one-line summary - 'three ways to manage data'.

SAY (read aloud)

Welcome, everyone. Today's session brings together three topics that look separate but are really one idea: managing data in Java. We'll look at three places data can live. First, in memory while your program runs - that's the Collections framework. Second, the way objects refer to and depend on each other in your code - that's class relationships. And third, on disk so the data survives after the program ends - that's File Input/Output. By the end you'll have the complete toolkit for handling data in any Java application, including every Spring Boot project you will build.

DELIVERY TIP

Keep this slide up for about 30 seconds. Set the expectation that this is theory-heavy, but every concept maps to code they will write later.

Slide 2 - Agenda

ON SCREEN

Five numbered items with icons: Collections framework (Lists, Sets, Maps); Sorting and choosing; Class relationships; Abstract vs Interface; Basic File I/O.

SAY (read aloud)

Here is our path for the session. We start with the Collections framework - Lists, Sets, and Maps, the three families you will use constantly. Then we look at how to sort collections and how to choose the right one for a job. Next we switch to class relationships - the difference between IS-A and HAS-A, which is one of the most important design ideas in object-oriented programming. We then compare abstract classes and interfaces. And we finish with basic File I/O - reading and writing files on disk. One thread ties all of this together: every topic today is about managing and organising data.

DELIVERY TIP

Mention that the code snippets in the deck are short on purpose - they are reading material to recognise, not exercises to copy.

Part 1 - The Collections Framework

Slide 3 - Why collections exist

ON SCREEN

Three cards: store 500 names; remove the item at index 200; and check if 'Ravi' is already present.

SAY (read aloud)

Before we use collections, let's understand the problem they solve. Think about plain arrays.

Scenario one: you create a String array for 500 names, and then you need a 501st. You're stuck - an array has a fixed size you must decide up front. Scenario two: you want to remove the element at index 200. With an array you have to manually shift every element after it one place to the left.

With a collection it is a single method call - list dot remove. Scenario three: you want to know whether 'Ravi' is already in the list. With an array you write a loop; with a collection you just call contains. So the takeaway is simple: arrays are fixed-size and have no built-in utilities. Collections are resizable, feature-rich, and type-safe through generics. They exist to solve exactly these three pains.

Slide 4 - The Collections hierarchy

ON SCREEN

A colour-coded tree: Iterable at the top, then Collection branching into List, Set, and Queue. Map is shown separately in its own box.

SAY (read aloud)

This is the map of the whole framework, so let's read it top to bottom. At the very top is the Iterable interface - anything iterable can be used in a for-each loop. Below it is Collection, the root of most of what we use. Collection branches into three: List, which is ordered, index-based, and allows duplicates; Set, which forbids duplicates; and Queue, which deals with items at the ends. Each of those has concrete classes - ArrayList and LinkedList for List, HashSet and TreeSet for Set, and so on. Now notice the box on its own at the bottom: Map. This is the one thing to lock in from this slide.

EMPHASISE

Map is NOT a subtype of Collection. It stores key-value pairs rather than individual elements, which is exactly why it lives in a separate hierarchy. Say this explicitly - it is a very common interview question.

Slide 5 - Generics

ON SCREEN

Two code panels: a raw ArrayList with no type that fails at runtime, versus a typed ArrayList that catches the error at compile time. Three benefit cards below.

SAY (read aloud)

Every collection uses generics, so let's give them one slide. Look at the left panel - the old style. We make an ArrayList with no type. We add the String 'Ravi', and then we accidentally add the number 101. The compiler says nothing. Later we pull element one out as a String and the program crashes at runtime with a ClassCastException. The bug hid until production. Now the right panel - the same code, but we declare ArrayList of String. The moment we try to add 101, the compiler stops us. The mistake is caught immediately, while we are still typing. That gives us three wins, shown in the cards: compile-time type checking; no casting needed, because the compiler already knows the type; and self-documenting code, because the angle-bracket String states our intent right there.

EMPHASISE

The rule: always specify the type parameter. The empty diamond brackets let the compiler infer it. Raw types only exist for backward compatibility with code older than Java 5.

Slide 6 - ArrayList, the default List

ON SCREEN

Definition of a List, then two panels: everyday operations on the left, a small performance table on the right. A yellow gotcha box at the bottom.

SAY (read aloud)

Let's start with Lists. A List keeps insertion order, lets you access elements by index, and allows duplicates. The workhorse implementation is ArrayList. Internally it is backed by a plain array. It starts with room for about ten elements, and when it fills up it creates a new array about one-and-a-half times larger and copies everything across. That copying is invisible to you, but worth knowing about. On the left are the everyday operations: add, add at a specific index, get, set to replace, contains, and remove - either by value or by index. The performance table on the right is the key story. Getting by index is constant time, very fast. Adding at the end is fast on average. But adding or removing in the middle is order-n, because everything after the insertion point must shift. And contains is order-n because it scans. So use ArrayList when you read often and modify rarely - which is most of the time. It is your default list.

EMPHASISE

Read out the gotcha: for an ArrayList of Integer, calling remove with the number 1 removes the element AT INDEX 1, not the value 1. To remove the value, wrap it: remove of Integer dot valueOf 1. This trips up almost everyone once.

Slide 7 - LinkedList, nodes and a Deque

ON SCREEN

A diagram of three linked nodes with prev and next pointers and HEAD/TAIL labels. Two panels: queue and stack operations, and a performance table.

SAY (read aloud)

LinkedList is the other main List, and it is built completely differently. Instead of one big array, it is a chain of nodes. Each node holds the data plus a reference to the previous node and the next node. The list itself only remembers the head and the tail. Because of that structure, inserting in the middle just re-points two links - there is no shifting of other elements. Now the performance table shows the trade-off. Getting an element by index is order-n, because you have to walk the chain node by node from the head. But adding or removing at the head or the tail is constant time - you just update a pointer. The cost is memory: every element carries two extra references. LinkedList is also a Deque - a double-ended queue - so it gives you queue operations like offer, poll, and peek, and stack operations like push and pop, shown in the left panel.

EMPHASISE

Modern advice on the slide: for a pure queue or stack, prefer ArrayDeque - it has better cache performance. Reach for LinkedList only when you genuinely need List and Queue behaviour at the same time.

Slide 8 - ArrayList vs LinkedList

ON SCREEN

A decision table comparing the two across random access, insertion at beginning and end, memory, and cache performance, with a Winner column.

SAY (read aloud)

Let's settle the rivalry with this decision table. Random access by index: ArrayList is constant time, LinkedList is order-n - ArrayList wins. Insert or delete at the beginning: LinkedList is constant time while ArrayList has to shift everything - LinkedList wins here. Insert at the end: both are effectively constant time - it is a tie. Memory per element: ArrayList is leaner, because LinkedList stores two extra references per node - ArrayList wins. Cache performance: ArrayList stores everything in a contiguous block so the CPU cache loves it, while LinkedList's scattered nodes do not - ArrayList wins again.

EMPHASISE

The practical conclusion on screen: use ArrayList by default. Only reach for LinkedList when you constantly add and remove at the ends or need a queue. In real code, ArrayList is the right answer about ninety percent of the time.

Slide 9 - Sorting: Comparable vs Comparator

ON SCREEN

Two panels: Comparable implemented inside the Employee class with compareTo, and Comparator written outside as lambdas for different orderings.

SAY (read aloud)

Java cannot guess how to sort your own objects - you have to tell it. There are two ways, shown side by side. On the left is Comparable. The class itself implements Comparable and provides a compareTo method that defines its natural order. Here Employee compares by salary. The contract is: return a negative number if this object comes before the other, zero if they are equal, and a positive number if it comes after. The handy trick is to use Double dot compare rather than subtracting, which avoids overflow bugs. Then Collections dot sort just works. On the right is Comparator. This lives outside the class and is far more flexible. With a lambda you can sort by name, or by salary descending using reversed, or by department and then by salary using comparing and thenComparing. You can have as many orderings as you like.

EMPHASISE

Which to use: Comparable when there is one obvious natural order and you own the class. Comparator when you need several orderings, or you need to sort a class you cannot modify, like String or Integer with custom logic.

Slide 10 - The Collections utility class & list creation

ON SCREEN

Left panel: static helpers from java.util.Collections. Right panel: a small table comparing Arrays.asList, List.of, and new ArrayList. A yellow box warns about Collections vs Collection.

SAY (read aloud)

Two practical helpers here. First, on the left, the Collections utility class - that is Collections with an s. It is a toolbox of static methods that work on any list: sort, reverse, shuffle, max, min, frequency to count occurrences, and unmodifiableList to get a read-only view. Very useful, and many freshers never discover it. Second, on the right, three ways to create a list, because they behave differently. Arrays dot asList gives you a fixed-size list - you can change existing elements but you cannot add or remove. List dot of, from Java 9, is truly immutable - you cannot change anything at all. And new ArrayList is fully mutable. If you have an immutable one and need to modify it, wrap it: new ArrayList passing List dot of inside.

EMPHASISE

Read the warning: Collections, plural, is the utility class. Collection, singular, is the interface. They are two different things - know the difference, it is a classic exam trap.

Slide 11 - Set & how HashSet works

ON SCREEN

Definition of a Set, a panel showing membership in O(1), and a panel explaining the hashing mechanism. A yellow box on the hashCode>equals contract.

SAY (read aloud)

Now Sets. A Set has two defining rules: no duplicates, and no index-based access. It is perfect for the 'have I seen this already?' question. With a List, checking membership means scanning the whole thing - order-n. With a Set it is effectively instant - order-1. How does HashSet pull that off? On add, it calls the object's hashCode method to get a number, maps that number to a bucket, and stores the element there. On contains, it computes the same hash, jumps straight to that one bucket, and checks only there. That is why lookups are instant no matter how big the set is. It also explains something students find strange: the iteration order is unpredictable, because elements sit in hash-order buckets, not in the order you added them.

EMPHASISE

Critical rule in the yellow box: for your own custom objects in a HashSet, or as HashMap keys, you must override BOTH hashCode and equals, consistently. If two objects are equal, they must return the same hashCode. If you override one, always override the other - otherwise lookups silently fail.

Slide 12 - Choosing a Set implementation

ON SCREEN

Three cards - HashSet, LinkedHashSet, TreeSet - a TreeSet code example, and a performance table.

SAY (read aloud)

Three flavours of Set, and choosing between them is about ordering. HashSet is the fastest - constant-time operations - but gives no order at all. It is the default. LinkedHashSet keeps insertion order by threading a linked list through the entries, at a small memory cost. Use it when you want no duplicates but predictable iteration order. TreeSet keeps everything sorted, using a balanced tree internally, so its operations are order-log-n - a bit slower - but it stays sorted automatically and adds navigation methods like first, last, floor, and ceiling. The little code example shows it nicely: add some numbers including a duplicate, print the TreeSet, and you get them in sorted order with the duplicate dropped.

DELIVERY TIP

If they only remember one line: HashSet for speed, LinkedHashSet for insertion order, TreeSet for sorted order.

Slide 13 - Map & HashMap

ON SCREEN

Definition of a Map, a HashMap code example with put, get, getOrDefault, containsKey, and a note box with the frequency-counter pattern.

SAY (read aloud)

Maps are the most useful collection in real projects, so pay attention here. A Map stores unique keys, each mapped to exactly one value. Think of a dictionary - word to definition - or a phone book - name to number - or student ID to grade. HashMap uses the very same hashing trick as HashSet, so lookups are constant time. In the example we put Mumbai with a population, then put Mumbai again with a new number - notice that updates the existing key rather than adding a second one, because keys are unique. Get returns the value, or null if the key is absent. getOrDefault is safer - it returns a fallback instead of null. And containsKey checks for a key. The note box shows a pattern you will use everywhere - a frequency counter. For each word, put the word with getOrDefault of that word, zero, plus one. That one line counts occurrences cleanly.

Slide 14 - Iterating Maps

ON SCREEN

Two panels: entrySet (and forEach) on the left, keySet/values on the right with a 'don't do this' warning. A small table of HashMap, LinkedHashMap, TreeMap ordering.

SAY (read aloud)

Iterating a Map trips people up, so here are the ways. The best one, on the left, is entrySet - it gives you the key and the value together in a single pass. Even cleaner in modern Java is forEach with a lambda that takes the key and value directly. On the right are keySet, when you only need the keys, and values, when you only need the values. But read the warning: do not loop over keySet and then call get inside the loop. That is two lookups for every row - wasteful. Use entrySet, which hands you both at once. And the bottom table mirrors what we saw with Sets: HashMap has no order and is fastest, LinkedHashMap keeps insertion order, and TreeMap keeps keys sorted at order-log-n cost.

Part 2 - Class Relationships

Slide 15 - Inheritance (IS-A)

ON SCREEN

Two panels: inherit-and-override with Animal and Dog, and polymorphism plus the super keyword.

SAY (read aloud)

Now we shift from storing objects to how objects relate to each other. The first relationship is IS-A - inheritance. A subclass is a kind of its superclass. A Dog is an Animal, so Dog extends Animal and inherits its behaviour, like eat, and can override it or add new behaviour like bark. Why does this matter? It enables polymorphism, shown on the right. We can write Animal a equals new Dog, because a Dog is an Animal, and when we call a dot eat, Java runs Dog's version at runtime. That is powerful - we can treat many types uniformly through their common parent. The super keyword also appears: super in parentheses calls the parent's constructor, and super dot methodName calls the parent's version of a method you have overridden, so you can reuse parent logic and add to it.

EMPHASISE

Two rules to stress. One: always write the `@Override` annotation - without it, a small typo in the method signature silently creates a brand-new method instead of overriding, and the bug is invisible.

Two: use inheritance only for a genuine, stable IS-A relationship, not just to reuse code - reuse is what composition is for, which is the next slide.

Slide 16 - Composition vs Aggregation

ON SCREEN

Two columns: Composition with House creating a Room (strong ownership), and Aggregation with Department referencing Employees (weak ownership).

SAY (read aloud)

Both of these are HAS-A relationships - one object holds another - but the difference is ownership strength, and it is the thing freshers most often confuse. On the left is composition - strong ownership. The House creates its own Room inside its constructor. The Room belongs entirely to the House; if the House is destroyed, the Room goes with it. A Room without a House makes no sense. That is composition. On the right is aggregation - weak ownership. The Department holds a list of Employees, but the Employees existed before being added and continue to exist if the Department is disbanded. An Employee can move to another department. The Department references them but does not own their lifecycle.

EMPHASISE

Give them the one-question test on screen: can the contained object exist and make sense WITHOUT the container? If yes, it is aggregation. If no, it is composition.

Slide 17 - Favour composition over inheritance

ON SCREEN

A problem statement about a Logger needing to both write a file and connect to a database, then a composition solution, and four benefit cards.

SAY (read aloud)

Here is a design principle you will hear throughout your career: favour composition over inheritance. Let's see why with a concrete case. Suppose you want a Logger that both writes to a file and connects to a database. With inheritance you would want to extend both `FileWriter` and `DatabaseConnector` - but Java has no multiple inheritance, so you simply cannot. Composition solves it instantly. The Logger HAS-A `FileWriter` and HAS-A `DatabaseConnector` as fields, and its `log` method just delegates to both. The four cards list the benefits: no inheritance limits, you can compose as many collaborators as you need; behaviour is swappable at runtime - inject a different writer; classes stay small and focused; and it is far easier to test, because you can inject mock dependencies.

EMPHASISE

The rule on screen: inherit for a genuine IS-A relationship; compose to reuse behaviour. Most of the time, composition is the right answer.

Slide 18 - Abstract class vs Interface

ON SCREEN

Two code panels - an abstract Vehicle class and a Saveable interface with multiple implements - and a comparison table.

SAY (read aloud)

Both abstract classes and interfaces let you define a contract, but they serve different purposes. On the left, an abstract class. Vehicle provides a shared, fully-written start method, but also declares an abstract mileage method that every subclass must implement. So an abstract class is a partial implementation - some behaviour given, some left to the subclass. You cannot create one directly with new. On the right, an interface. Saveable defines a save method, and since Java 8 it can also provide default methods with a body. The big advantage is that a class can implement many interfaces at once - here Report implements both Printable and Saveable - which is how Java works around the lack of multiple inheritance. The table summarises the differences: abstract classes can have constructors and instance fields, interfaces basically cannot; both can mix abstract and concrete methods, but only interfaces allow multiple inheritance.

EMPHASISE

Which to use: an abstract class when classes share state and behaviour; an interface to define a capability that unrelated classes might share - like Comparable, Runnable, or Serializable.

Part 3 - Basic File I/O

Slide 19 - Why File I/O

ON SCREEN

The persistence bridge explained, two columns for byte streams and character streams, and a note on the File class.

SAY (read aloud)

We have kept data in memory with collections. Now we put it on disk with File Input/Output. Why files? Because they keep data alive after the program ends, they need no server, and they are perfect for configuration, logs, and exports. Java gives you two families of streams. Byte streams - InputStream and OutputStream - are for binary data like images, audio, and PDFs. Character streams - Reader and Writer - are for text, and they handle character encoding for you. The rule of thumb: you will work with text about ninety percent of the time, so reach for character streams. One more thing in the note: the File class represents a path only - it can check whether a file exists, its size, and create or delete files and folders, but it does not read or write the content itself. And always use File dot separator or the Paths helper instead of hardcoding slashes, so your code works on both Windows and Linux.

Slide 20 - Buffered text I/O

ON SCREEN

Two panels: writing with `BufferedWriter/PrintWriter`, and reading line by line with `BufferedReader`. A key box on the wrapping idea.

SAY (read aloud)

This is the practical way to read and write text files. The idea is wrapping - you take a basic `FileReader` or `FileWriter` and wrap it in a buffered layer that reads or writes large chunks at once instead of one slow character at a time. On the left, writing. We wrap a `FileWriter` in a `BufferedWriter`, and then in a `PrintWriter`, which adds the convenient `println` and `printf` methods you already know from the console. Notice it is inside a `try-with-resources`, so the file is flushed and closed automatically. On the right, reading. We wrap a `FileReader` in a `BufferedReader` and call `readLine` in a while loop. `readLine` returns null at the end of the file, which ends the loop naturally. Then we split each line on commas to pull out the fields. The key box explains the layering: `BufferedReader` wraps `FileReader` wraps the file, and each layer adds a capability - `FileReader` opens it, `BufferedReader` adds buffering and `readLine`.

DELIVERY TIP

Mention `Scanner` as the easy alternative for token-based reading - `nextInt`, `nextLine` - but note it is slower for large files.

Slide 21 - try-with-resources, NIO.2 & Properties

ON SCREEN

Two panels: safe auto-closing with `IOException` handling, and the concise NIO.2 Files API. A note box with the Properties file pattern.

SAY (read aloud)

Three finishing touches on File I/O. First, on the left, always auto-close. A file stream is a real operating-system resource; if you forget to close it, you leak the handle and may even lock the file. The fix is `try-with-resources` - declare the stream in the `try` parentheses and Java closes it for you, even if an exception is thrown. And always catch `IOException` to handle problems like a missing file or no permission. Second, on the right, the modern API: `java dot n-i-o dot file`. Look how short it is - `Files dot writeString` to write, `Files dot readString` to read the whole file, `Files dot readAllLines` to get a list of lines. For new code, prefer this; for older code and frameworks, you will still see the `java dot i-o stream` style. Third, the note box - the Properties class. You load a properties file and then call `getProperty` by key. This matters because it is almost exactly how Spring Boot reads its `application dot properties` file. When Spring reads your database URL, it is doing something very close to this.

EMPHASISE

Tie it back: 'You have already seen the innards of Spring Boot's configuration system - it is just `Properties dot load` under the hood.'

Slide 22 - Collections + File I/O in one flow

ON SCREEN

A single larger code block: read a CSV into a List of Employee, sort and group with Collections, then write the result back to a new file.

SAY (read aloud)

This slide is the payoff - it shows everything working together in one realistic workflow. Step one: read. We open the CSV with a `BufferedReader`, skip the header line, then loop with `readLine`, split each line on commas, and build an `Employee` object from the parts, adding each to an `ArrayList`. Step two: process. We sort the list with a `Comparator` by salary descending, and we build a `HashMap` that counts employees per department using `merge`. Step three: write. We open a `PrintWriter` wrapped around a `BufferedWriter` and a `FileWriter`, print a header, and `printf` each employee back out to a new report file. In one short program we have used `ArrayList`, `sort`, `Comparator`, `HashMap`, `BufferedReader`, `BufferedWriter`, and `PrintWriter` - the entire session, cooperating to read, transform, and persist data.

Recap & Close

Slide 23 - Session recap

ON SCREEN

A six-cell grid: Lists, Sets, Maps, Class relationships, File I/O, and Golden rules.

SAY (read aloud)

Let's recap in one view. Lists: `ArrayList` is your default with fast index access; `LinkedList` is fast at the ends and doubles as a queue; `Comparable` defines natural order while `Comparator` is flexible. Sets: `HashSet` is fastest with no order, `LinkedHashSet` keeps insertion order, `TreeSet` keeps things sorted. Maps: `HashMap` is fast and unordered, `TreeMap` is sorted, iterate with `entrySet`, and override `hashCode` and `equals` together for custom keys. Class relationships: IS-A is inheritance, HAS-A is composition or aggregation, favour composition for reuse, abstract classes are partial implementations while interfaces are pure contracts. File I/O: use `BufferedReader` and `BufferedWriter` for text, `PrintWriter` for formatting, the `n-i-o-dot-2 Files` class for concise modern code, and `Properties` for config. And the golden rules: always use generics, `ArrayList` and `HashMap` are the right default about ninety percent of the time, and always use `try-with-resources` for any stream.

Slide 24 - Closing

ON SCREEN

The closing blue slide: 'The complete data toolkit.'

SAY (read aloud)

And that is the session. You now have the complete toolkit for managing data in Java - in memory with Collections, structurally with class relationships, and on disk with File I/O. Here is the thing to carry forward: in every Spring Boot project you build, all three of these are in play every single day. The List that holds your query results, the relationships between your entity classes, the properties file Spring reads at startup - it is all what we covered today. Thank you. Let's take any questions.

DELIVERY TIP

Leave this slide up during Q&A. Common questions: 'when exactly would I use LinkedList?' and 'why override both hashCode and equals?' - both are answered on earlier slides you can flip back to.